Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Proyecto #3

M.U.M.N.K8s
(Monitoreo de Unidades Militares en la Nube con Kubernetes)

PONDERACIÓN: 50pts

Tiempo estimado: 60hrs

Índice

1. Resumen Ejecutivo **3**
2. Competencia que desarrollaremos **3**
3. Objetivos del Aprendizaje **3**
3.1 Objetivo General 3
3.2 Objetivos Específicos 3
4. Enunciado del Proyecto **5**
4.1 Descripción del problema a resolver 5
4.2 Alcance del proyecto 5
4.4 Entregables 6
5. Metodología **8**
6. Desarrollo de Habilidades Blandas **8**
6.1 Proyectos Individuales **9**
7. Cronograma **10**
8. Rúbrica de Calificación **11**
8.1 Requisitos para optar a la calificación 11
8.2 Resumen de Puntuaciones **12**
8.3 Detalle de la Calificación **12**
8.4 Valores 14
8.5 Comentarios Generales 14

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Mejorar y entregar nuestros trabajos como parte de las responsabilidad del curso

1.2. Competencia(s)

Indique la competencia general del curso y la competencia específica trabajada en esta lectura.

Tipo de Competencia	
Competencia General	Aprender el uso y funciones de un SO
Competencia Específica	Aprender Linux a fondo y las máquinas virtuales para su suo

1.3. Objetivo SMART

Define el/los objetivo(s) que se pretende cumplir con la tarea planteada, con un enfoque SMART, toma en consideración que un objetivo SMART, va enfocado a la habilidad(y no a tanto a la teoría

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Implementar una arquitectura de sistema distribuido y escalable en Google Cloud Platform (GCP) que procese reportes datos en tiempo real utilizando GKE, Rust, Go, RabbitMQ, Valkey y Grafana.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	El sistema debe soportar alta carga de datos mediante HPA (1-3 réplicas con CPU > 30%) y visualizar el 100% de las métricas requeridas en dashboards de Grafana en tiempo real, simulando un sistema distribuido.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando contenedores Docker publicados en Zot, orquestación en GKE.
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	con habilidades prácticas que fortalezcan la formación en informática y sistemas, preparándose para cursos más avanzados en virtualización y orquestación utilizando containers.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Cumplir con el aprendizaje y la entrega final en un plazo máximo de 04 semanas.

2. Resumen Ejecutivo

El proyecto "M.U.M.N.K8s" tiene como propósito aplicar los conocimientos adquiridos en las unidades 1 y 2 del laboratorio, enfocándose en la implementación de una arquitectura de sistema distribuido y escalable en Google Cloud Platform (GCP) utilizando Google Kubernetes Engine (GKE).

Se construirá un sistema que simula la recepción y procesamiento de reportes militares generados en tiempo real por distintos países. Cada reporte representará el estado parcial de recursos bélicos, incluyendo la cantidad de aviones de guerra en el aire y barcos de guerra en el mar, junto con su país de origen y marca de tiempo.

Este sistema involucrará la generación de tráfico con Locust, una API REST en Rust, servicios en Go para procesamiento y comunicación gRPC, RabbitMQ como sistema de mensajería, consumidores para procesar los mensajes, y bases de datos/servicios de visualización desplegados y administrados mediante un **Operator de Kubernetes construido con Operator SDK (Helm-based operator)**. Dicho Operator deberá encapsular a **Valkey** y a **Grafana** como recursos personalizados (Custom Resources) dentro del clúster, de forma que su instalación, configuración y ciclo de vida se gestionen de manera declarativa (en lugar de máquinas virtuales). Todas las imágenes Docker de los componentes deberán publicarse y consumirse desde Zot, desplegado en una máquina virtual de GCP fuera del clúster.

El proyecto busca demostrar la comprensión y aplicación de tecnologías de contenedores, orquestación, **extensión de Kubernetes mediante Operators (CRDs y controladores basados en Helm)**, comunicación entre microservicios, manejo de concurrencia, visualización de métricas y pruebas de carga, así como la integración opcional de Dapr como mecanismo alternativo para el envío de mensajes.

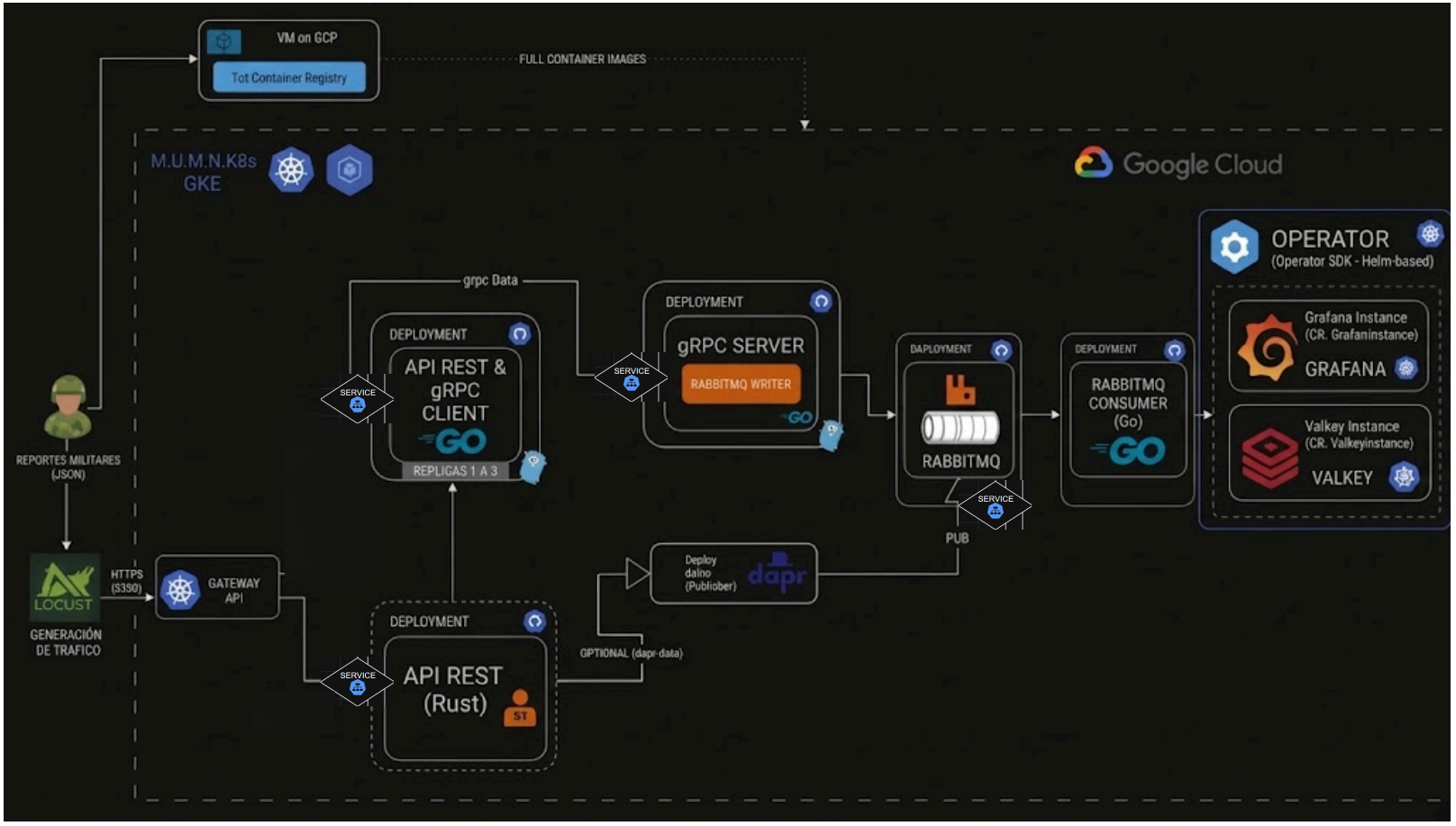
3. Enunciado del Proyecto

En la actualidad, los sistemas distribuidos deben ser capaces de procesar grandes volúmenes de eventos en tiempo real, garantizar escalabilidad, tolerancia a fallos y observabilidad, y además integrarse con distintos mecanismos de comunicación y almacenamiento. En este proyecto se simulará un escenario de monitoreo bélico donde continuamente se generan reportes militares desde distintos países.

Cada evento contendrá información resumida sobre unidades estratégicas, por ejemplo: país, cantidad de aviones de guerra en el aire, cantidad de barcos de guerra en el mar y una marca de tiempo. El desafío consiste en diseñar e implementar una arquitectura distribuida, escalable y resiliente que permita recibir, procesar, enrutar, consumir, almacenar y visualizar dichos datos utilizando tecnologías modernas de nube, contenedores y virtualización.

La solución deberá ejecutarse en GCP utilizando instancias N1, un clúster de GKE, Kubernetes Gateway API para exponer el sistema, una API REST en Rust, servicios gRPC en Go, RabbitMQ como broker principal de mensajería, y un Operator personalizado (Operator SDK, Helm-based) que despliegue y administre tanto Valkey como Grafana dentro del clúster, cada uno como un recurso personalizado (Custom Resource) independiente. Adicionalmente, el estudiante podrá implementar un flujo alternativo de mensajería utilizando Dapr como parte extra del proyecto.

4.3 Alcance del proyecto



- **Componentes clave incluyen:** Locust (generador de carga), Kubernetes Gateway API, API REST (Rust), servicios gRPC (Go), RabbitMQ, consumidores (Go), un Operator (Operator SDK - Helm) que administra Valkey y Grafana como Custom Resources dentro del clúster, y Zot como Container Registry externo al clúster.

4.3 Alcance del proyecto

- **Alcance obligatorio:**
  - **Locust:** Generación de tráfico con la estructura JSON especificada hacia el endpoint público expuesto mediante **Kubernetes Gateway API**.
    Los datos enviados deberán simular reportes militares con valores aleatorios dentro de rangos definidos.
  - **Gateway API:** Exposición del sistema utilizando Kubernetes Gateway API en sustitución del uso de Ingress Controller.
    El sistema deberá contar con las rutas para:
    - `/grpc-#carnet`
    - `/dapr-#carnet` (parte opcional con Dapr)
  - **Deployments de Rust:** API REST que recibe peticiones de Locust, envía a un Deployment de Go, soporta alta carga y escala con HPA (1-3 réplicas, CPU > 30%).
  - **Deployments de Go:**
    - **Deployment 1 (API REST y gRPC Client):** Recibe de Rust, actúa como cliente gRPC, invoca funciones para publicar en RabbitMQ. Es decir un deployment con 2 containers.
    - **Deployments 2 y 3 (gRPC Server y Writer RabbitMQ):** recibe solicitudes gRPC y publica mensajes en RabbitMQ. Deben realizarse pruebas con 1 y 2 réplicas en los componentes que la cátedra defina para análisis de rendimiento. Es decir un deployment con 2 containers.
  - **RabbitMQ:** Broker principal de mensajería del proyecto. Será el único sistema de colas obligatorio utilizado para el flujo principal.
  - **RabbitMQ Client (Consumer) (Deployment):** Deployment encargado de consumir los mensajes de RabbitMQ, procesarlos y almacenar la información resultante en Valkey.
  - **Operator para Valkey (Operator SDK - Helm-based):** El estudiante deberá construir un **Operator de Kubernetes** siguiendo la guía oficial de Operator SDK para operadores basados en Helm (https://sdk.operatorframework.io/docs/building-operators/helm/quickstart/).
    Este Operator deberá:
    - Definir un **Custom Resource Definition (CRD)** (por ejemplo, `ValkeyInstance` o similar) que represente el despliegue de Valkey.
    - Encapsular en un **Helm chart** la configuración necesaria de Valkey (Deployment/StatefulSet, Service, PersistentVolumeClaim para persistencia).
    - Reconciliar el estado deseado del Custom Resource contra el clúster (crear/actualizar/eliminar los recursos de Valkey al crear/actualizar/eliminar el Custom Resource correspondiente).
    - Garantizar persistencia de datos (mediante PVC) y conectividad entre los Consumers y el Service de Valkey generado por el Operator.
    - Ser instalado y ejecutado dentro del clúster de GKE (no como máquina virtual).
  - **Operator para Grafana (Operator SDK - Helm-based):** El mismo Operator (o un segundo *kind* dentro de él) deberá administrar el despliegue de **Grafana** como un Custom Resource independiente (por ejemplo, `GrafanaInstance`). Este componente del Operator deberá:

- Encapsular en un Helm chart la configuración de Grafana (Deployment, Service, datasource/provisioning de dashboards).
- Conectarse a la fuente de datos correspondiente (Valkey, directamente o mediante un adaptador/exporter) para construir y mostrar los dashboards requeridos.
- Permitir que la creación de la instancia de Grafana se realice de forma declarativa mediante el Custom Resource, gestionado íntegramente por el Operator dentro del clúster de GKE.

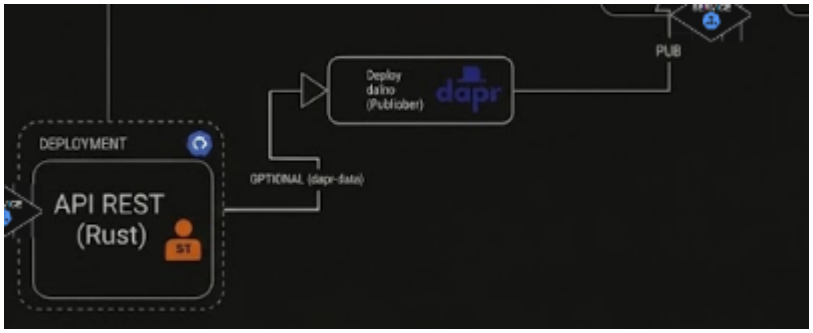
- **Zot:** Implementado en una VM de GCP fuera del clúster K8s. Todas las imágenes Docker de los componentes se publican y se descargan desde Zot. Usar HTTPS.
- **OCI Artifact:** Descarga de archivo de entrada desde el registry como un OCI Artifact (se debe especificar qué archivo y cómo se usa en la documentación)
- **Infraestructura en GCP o Local:** Todo el proyecto debe desplegarse en Google Cloud Platform, utilizando instancias N1.

- **Documentación:** El manual técnico deberá explicar:
  - arquitectura general,
  - flujo completo de datos,
  - configuración de Gateway API,
  - comunicación REST y gRPC,
  - uso de RabbitMQ,
  - diseño y funcionamiento del Operator (CRDs, Helm charts encapsulados, lógica de reconciliación) usado para desplegar Valkey y Grafana,
  - configuración de HPA,
  - publicación/consumo de imágenes desde Zot,
  - pruebas realizadas y conclusiones.

  El manual técnico deberá ser entregado exclusivamente en formato Markdown.
- **Sugerencias Generales:** Uso de namespaces, Gateway API, creación propia de imágenes Docker.
- **Requisitos Mínimos para tener derecho a calificación:**
  - Clúster de Kubernetes
  - Uso obligatorio de Locust
  - Uso obligatorio de GKE
  - Uso obligatorio de RabbitMQ
  - Uso obligatorio de operador para Valkey y Grafana
- **Restricciones:** Proyecto individual, uso obligatorio de Locust y GKE
  NO HABRÁ PRÓRROGA.
- **Github:** Repositorio privado, carpeta llamada ‘proyecto3’, agregar auxiliares.

- **Implementación con Dapr:** Se deberá implementar un flujo adicional de envío de mensajes utilizando Dapr, expuesto por medio de la ruta:
  - /dapr-#carnet

  Esta implementación podrá coexistir con el flujo base `/grpc-#carnet`, con el objetivo de comparar ambos enfoques a nivel de arquitectura, integración y comportamiento.



  Para la implementación con Dapr, deberá integrarse el SDK de Dapr en los servicios que actualmente utilizan gRPC, con el fin de habilitar:
  - El envío de información mediante pub/sub (publicador)
  - La recepción de información mediante suscripción a eventos
  - La comunicación desacoplada entre servicios a través de RabbitMQ

- **Recomendaciones:**
  - **Optimización del rendimiento:** Uso de `requests` y `limits` en los deployments para mejorar estabilidad del sistema.
  - **Expiración de datos:** Configuración de TTL o políticas de expiración en Valkey para evitar saturación de memoria.

Seguridad en Tiempo Real con Falco Para introducir los conceptos de seguridad en tiempo real y observabilidad de amenazas en contenedores, el estudiante podrá implementar la herramienta **Falco** dentro de su clúster de GKE. Falco audita el comportamiento del sistema a nivel de *syscalls* de Linux y Kubernetes.

- **Instalación:** Instalar Falco en el clúster de GKE (se recomienda el uso de Helm para facilitar el despliegue).
- **Regla Personalizada (Falco Rule):** Crear e implementar una (1) regla de Falco personalizada. La regla deberá detectar **la apertura o lectura de un archivo o directorio sensible específico** (por ejemplo, detectar intentos de lectura del archivo `/etc/shadow` o de un directorio confidencial de prueba como `/app/secretos`) dentro de cualquiera de los pods de la API REST en Rust o de los servicios en Go.
- **Simulación de la amenaza:** El estudiante deberá ingresar intencionalmente a uno de sus pods desde la consola (utilizando `kubectl exec`) y ejecutar

un comando de lectura (por ejemplo, `cat /etc/shadow` o `ls /app/secretos`) para forzar a que la regla intercepte la llamada al sistema y la alerta se dispare.

- **Evidencia en Documentación:** El archivo YAML de la regla creada y la captura de pantalla de los logs de Falco mostrando la alerta (generada tras el intento de lectura del archivo/directorio) deberán incluirse dentro del manual técnico en formato Markdown.

4.3.1 Estructura de los mensajes:

La estructura de los mensajes deberá representar reportes militares simulados. Un ejemplo de JSON válido es el siguiente:

```
{
 "country": "ESP",
 "warplanes_in_air": 42,
 "warships_in_water": 14,
 "timestamp": "2026-03-12T20:15:30Z"
}
```

Reglas de generación

- **`country`:** código de país de 3 letras.
- **`warplanes_in_air`:** número entero aleatorio dentro de un rango definido.
- **`warships_in_water`:** número entero aleatorio dentro de un rango definido.
- **`timestamp`:** fecha y hora del evento.

Rangos sugeridos

- **`warplanes_in_air`:** entre 0 y 50
- **`warships_in_water`:** entre 0 y 30

Propuesta de estructura gRPC

```
syntax = "proto3";
package wartweets;
option go_package = "./proto";

// Mensaje que se enviará
message WarReportRequest {
 Countries country = 1;
 int32 warplanes_in_air = 2;
 int32 warships_in_water = 3;
 string timestamp = 4;
}

// Lista de los únicos posibles países aceptados por el proyecto
enum Countries {
 countries_unknown = 0;
 usa = 1;
 rus = 2;
 chn = 3;
 esp = 4;
 gtm = 5;
}

// Respuesta del servidor
message WarReportResponse {
 string status = 1;
```

```
}

// Servicio gRPC
service WarReportService {
 rpc SendReport (WarReportRequest) returns (WarReportResponse);
}
```

4.3.2 Estructura del Dashboard Requerido

Asignación de país en base al último dígito de su carnet

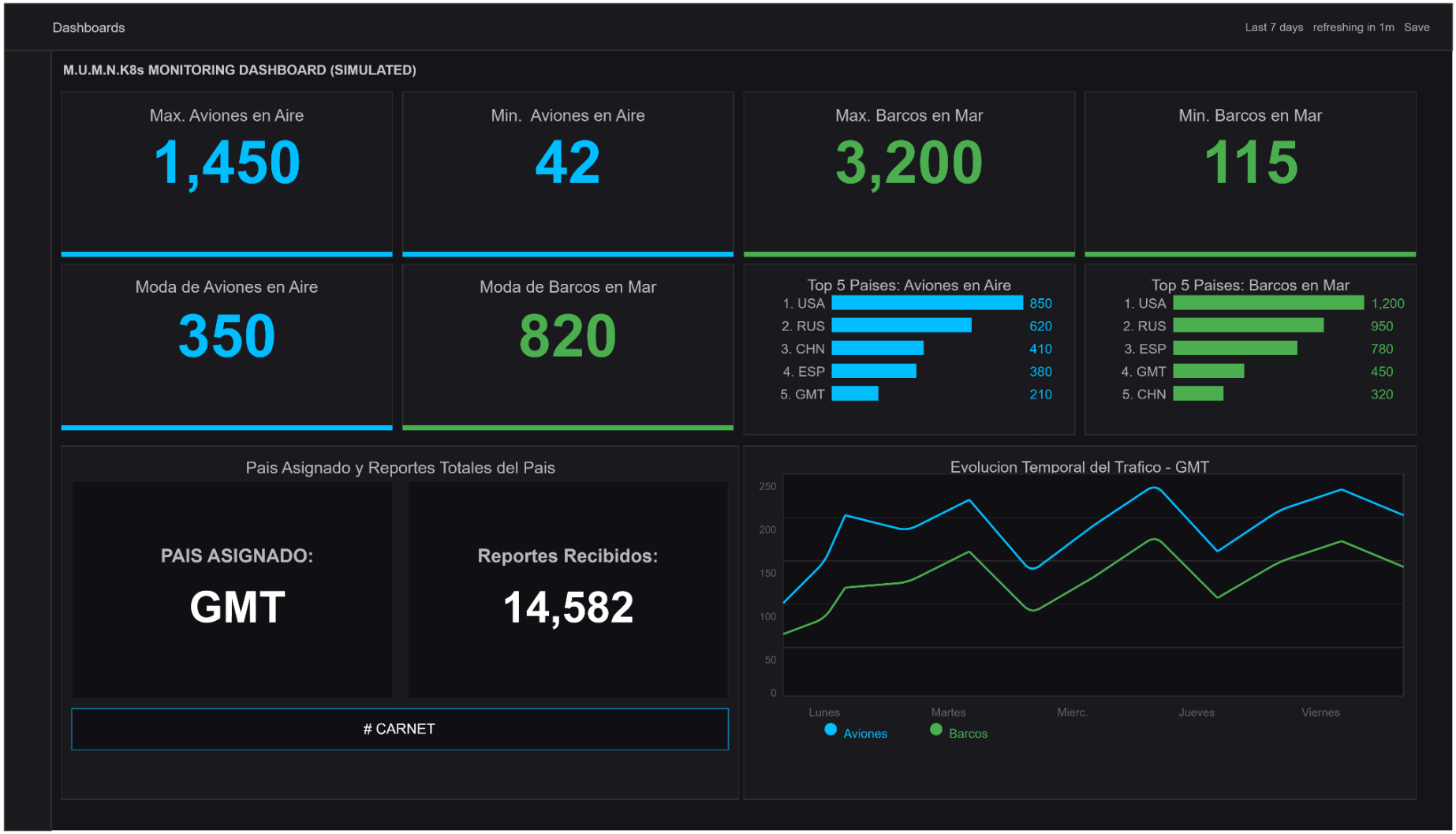
- **0,1** = USA
- **2,3** = RUS
- **4,5** = CHN
- **6,7** = ESP
- **8,9** = GTM

Visualizaciones requeridas

- Valor máximo general de aviones en aire
- Valor mínimo general de aviones en aire
- Valor máximo general de barcos en mar
- Valor mínimo general de barcos en mar
- Top de países con más aviones en aire
- Top de países con más barcos en mar
- Moda de aviones en aire
- Moda de barcos en mar
- Serie temporal del país asignado, mostrando la evolución de:
    - aviones en aire
    - barcos en mar

- Nombre del país asignado
- Cantidad total de reportes recibidos del país asignado

A continuación, se presenta un ejemplo ilustrativo del dashboard que se espera desarrollar en Grafana, mostrando su estructura y visualizaciones esperadas.



Gráfica en HD: https://drive.google.com/file/d/1xd_uKJ_6Q9pJ66fJwinSK-5CKvBwni_q/view?usp=sharing
(se recomienda descargar el archivo y abrirlo en https://draw.io)

3.3 Entregables

1. Código fuente en repositorio privado GitHub nombre
   UTILIZAR MISMO REPOSITORIO que en el Proyecto 1, y crear una carpeta para el Proyecto 3 llamada 'proyecto3'
   Deberá añadir a los auxiliares como colaboradores
   Usuarios de auxiliares: @JoseLorenzana272, @KINGR0X

2. Manual técnico y guía de instalación
3. Capturas de prueba y evidencia funcional

4. Material de apoyo

https://github.com/JoseLorenzana272/sopes1_ejemplos

5. Metodología

6. **Fase 1: Configuración e Infraestructura Base (Semana 1)**
   - 6.1. Configurar cuenta de GCP y crear el clúster de GKE utilizando instancias N1.
   - 6.2. Instalar y configurar Zot en una VM de GCP fuera del clúster.
   - 6.3. Desarrollar y contenerizar la API REST en Rust. Publicar la imagen en Zot.
   - 6.4. Desarrollar y contenerizar el Go Deployment 1 encargado de la comunicación inicial y del rol de gRPC Client. Publicar la imagen en Zot.
   - 6.5. Configurar Locust para generar tráfico básico con la estructura JSON definida para reportes militares.
   - 6.6. Desplegar y configurar Kubernetes Gateway API para exponer los servicios del sistema.
   - 6.7. Realizar pruebas iniciales del flujo:
     Locust → Gateway API → API Rust → Go Deployment 1
7. **Fase 2: Comunicación Interna y Publicación de Mensajes (Semana 2)**
   - 7.1. Desplegar RabbitMQ en GKE como broker principal de mensajería.
   - 7.2. Desarrollar y contenerizar el Go Deployment 2, encargado del rol de gRPC Server y publicación de mensajes en RabbitMQ. Publicar la imagen en Zot.
   - 7.3. Integrar el Go Deployment 1 con el servicio publicador mediante gRPC.
   - 7.4. Configurar la ruta principal del sistema para el flujo estándar:
     `/grpc-#carnet`
   - 7.5. Realizar pruebas de publicación de mensajes hacia RabbitMQ.
   - 7.6. Validar el flujo completo de recepción, transformación y envío de mensajes.
8. **Fase 3: Consumo de Mensajes y Persistencia en Máquina Virtual (Semana 2)**
   - 8.1. Desarrollar y contenerizar el Consumer en Go para procesar mensajes desde RabbitMQ. Publicar la imagen en Zot.

8.2. Con `operator-sdk init` y `operator-sdk create api` (plugin Helm), generar el esqueleto del Operator y del CRD para Valkey (por ejemplo, `ValkeyInstance`).
8.3. Construir/adaptar un Helm chart de Valkey y encapsularlo dentro del Operator (`helm-charts/valkeyinstance`).
8.4. Compilar, empaquetar y desplegar el Operator dentro del clúster de GKE (imagen del Operator publicada en Zot).
8.5. Crear el Custom Resource de Valkey y validar que el Operator reconcilia correctamente (Deployment/StatefulSet, Service, PVC).
8.6. Integrar el Consumer para almacenar los datos procesados en el Service de Valkey administrado por el Operator.
8.7. Realizar pruebas de consumo, almacenamiento y persistencia de los datos generados por el sistema.

9. **Fase 4: Visualización, Ruta Alternativa y Pruebas de Carga (Semana 3)**
9.1. Extender el Operator con un segundo *kind* (o un segundo Operator) para Grafana (por ejemplo, `GrafanaInstance`), encapsulando su Helm chart correspondiente.
9.2. Desplegar el Custom Resource de Grafana y validar que el Operator provisiona correctamente el Deployment, Service y datasources.
9.3. Configurar los dashboards requeridos para visualizar la información almacenada y procesada en Valkey.
9.4. Implementar, como parte de punteo extra, la ruta alternativa `/dapr-#carnet`.
9.5. Integrar y probar el envío de mensajes utilizando Dapr.
9.6. Realizar pruebas de carga completas con Locust.
9.7. Implementar y probar HPA para el deployment de Rust.
9.8. Analizar el rendimiento del sistema con 1 y 2 réplicas en los deployments de Go definidos para evaluación.
9.9. Comparar el comportamiento general del flujo estándar y del flujo con Dapr, en caso de haberse implementado.

10. **Fase 5: Documentación, Validación y Entrega (Semana 4)**
10.1. Redactar el manual técnico, incluyendo respuestas a las preguntas y análisis de rendimiento.
10.2. Verificar que todos los componentes estén correctamente configurados y documentados en el repositorio.
10.3. Validar que las imágenes publicadas en Zot puedan ser consumidas correctamente por los componentes del sistema.
10.4. Preparar la entrega final conforme a los requisitos establecidos.

6. Recursos y herramientas a utilizar

Listado de materiales que los estudiantes deberán usar o investigar:

- **Dapr sidecar:** https://docs.dapr.io/concepts/dapr-services/sidecar/
- **Operator SDK (Helm-based operators):** https://sdk.operatorframework.io/docs/building-operators/helm/quickstart/

7. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico, esta sección se centra en las habilidades blandas que los estudiantes deberán mejorar a lo largo del proyecto. Dado que este proyecto se plantea como individual, el enfoque es similar al Proyecto 1 pero con mayor énfasis en la gestión de complejidad.

- **6.1.1 Autogestión del Tiempo y Complejidad:** El estudiante deberá gestionar un proyecto con múltiples componentes interdependientes y tecnologías diversas, requiriendo una excelente planificación y priorización para cumplir con los plazos.
- **6.1.2 Responsabilidad y Compromiso Técnico:** Asumir la responsabilidad completa de una arquitectura compleja, desde el diseño conceptual hasta la implementación funcional y el análisis de rendimiento.
- **6.1.3 Resolución Avanzada de Problemas y Depuración Distribuida:** Identificar y solucionar problemas en un sistema distribuido, donde los errores pueden ser difíciles de rastrear a través de múltiples servicios y tecnologías.
- **6.1.4 Investigación y Aprendizaje Autónomo:** Investigar y aprender sobre múltiples tecnologías (Kubernetes, RabbitMQ, Go, Rust, Grafana, etc.) y sus mejores prácticas de implementación e integración.
- **6.1.5 Documentación Técnica y Comunicación de Resultados:** Elaborar un manual técnico claro y conciso que no solo describa el sistema, sino que también analice y justifique decisiones de diseño y compare el rendimiento de tecnologías.

8. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	24/09/2026	22/10/2026
Calificación	24/10/2026	24/10/2026

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Si/No)
Entrega Completa	Se entrega el enlace al repositorio privado de GitHub (con acceso a los auxiliares) conteniendo todo el código fuente, Dockerfiles, YAMLs de Kubernetes y la documentación técnica.	
Funcionalidad del Clúster en GCP (GKE)	El sistema está desplegado y es funcional en un clúster de GKE.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Conocimiento técnico (90%)		
Arquitectura General y Despliegue en GKE/Zot	5	
API Rest en Rust	5	
Servicios de Ingesta y Procesamiento (Go)	10	
Arquitectura General Despliegue en Zot	10	
Message Broker y Consumidor RabbitMQ	15	
Almacenamiento y Visualización (Valkey, Grafana)	45	
Sub-Total Conocimiento	**90**	
2. Análisis, Documentación y Cumplimiento (10%)		

Pruebas de Carga, HPA y Análisis de Réplicas	5	
Manual Técnico y Respuestas a Preguntas	5	
Sub-Total Habilidades	**10**	
TOTAL	**100**	

*La calificación debe incluir ambas áreas conocimientos y habilidades.

8.3 Detalle de la Calificación

Criterio	Descripción	Puntos Máximos	Puntuación Obtenida
1. Implementación funcional			
1.1 Arquitectura General Despliegue (Ingress Controller)	Implementación de Ingress en Kubernetes para la exposición y acceso al sistema desplegado	5	
1.2 API Rest en Rust	API REST/gRPC en Rust funcionando y escalando.	5	
1.3 Servicios de Ingesta y Procesamiento (Go)	Servicios en Go (API/gRPC Client, gRPC Server/Writers) implementados correctamente, con manejo de concurrencia y comunicación gRPC.	10	
1.4 Arquitectura General Despliegue en Zot	Despliegue completo y funcional de todos los componentes en GCP. Despliegue correcto de Zot en vm para imágenes.	10	
1.5 Message Broker y Consumidor RabbitMQ	RabbitMQ desplegado y configurado. El Writer publica mensajes correctamente. Los consumidores leen las colas/tópicos y procesan los mensajes.	15	
1.6 Almacenamiento (Valkey)	Valkey desplegado. Los consumidores almacenan datos en Valley.	20	

1.7 Grafana	Grafana desplegada. Visualización de los datos requeridos en Grafana.	25	
Sub-Total de Puntos		**90**	
2.Análisis, Documentación y Cumplimiento			
2.1 Pruebas de Carga, HPA y Análisis de Réplicas	Locust genera la carga especificada. la API Rust funciona según lo definido. Se realizan y documentan pruebas comparativas con 1 y 2 réplicas para Go Writers y Valkey, analizando rendimiento.	**5**	
2.2 Manual tecnico	El manual es claro, completo, incluye todas las secciones requeridas (despliegue, arquitectura, pruebas de funcionalidad).	**2**	
2.3 Respuestas a Preguntas	Se responden de forma correcta y justificada todas las preguntas planteadas durante la calificación.	**3**	
Sub-Total de Puntos		**10**	
Total		**100**	

8.5 Restricciones para tener derecho a nota del Proyecto:

IMPORTANTE LEER DETENIDAMENTE

La nota final del proyecto está sujeta al cumplimiento obligatorio de los siguientes 2 requisitos. La omisión de cualquiera de ellos dejará al estudiante con una nota de 0 puntos sobre su proyecto, independientemente del avance logrado en otras áreas. Dichos requisitos serán verificados durante el proceso de calificación.

- **Entrega en UEDI *(independientemente de su auxiliar asignado)***
- **Entrega del Manual Técnico en formato *Markdown* dentro del repositorio de GitHub**

Criterios de evaluación y penalización:

Además de lo anterior, la ausencia de los siguientes elementos técnicos afectará la nota final con los porcentajes de penalización indicados:

Elemento técnico	**Penalización por incumplimiento**
Uso de RabbitMQ	`- 30% de la nota final`
Uso de Locust	`- 10% de la nota final`
Uso de API Gateway	`- 20% de la nota final`
Uso de Zot	`- 05% de la nota final`
Demostración de conocimientos prácticos	`- 15% de la nota final`

Detalle de la Calificaciones

RÚBRICA DE CALIFICACIÓN - PROYECTO 1						
No.	**Criterio de evaluación**	**Punteo máximo**		**Satisfactorio (100%-61%)**	**Necesita mejorar (60%-0%)**	**Obtenido**
CONOCIMIENTOS						
1	**Implementación funcional**					
1.1	**Arquitectura General Despliegue (Kubernetes Gateway API)**	5	Descripción	Implementación de Gateway API en Kubernetes para la exposición y acceso al sistema desplegado	El Kubernetes Gateway API no funciona correctamente	
			Rango del punteo	5-3	2-0	
1.2	**API Rest en Rust**	5	Descripción	API REST/gRPC en Rust funcionando y escalando.	La api no fue desarrollada en Rust. La api no funciona correctamente o no contiene todos los endpoints especificados	
			Rango del punteo	5-3	2-0	
1.3	**Servicios de Ingesta y Procesamiento (Go)**	10	Descripción	Servicios en Go (API/gRPC Client, gRPC Server/Writers) implementados	Servicios no desarrollados en Go. Los servicios no funcionnan correctamente. Falto el manejo de concurrencia y/o la comunicación	

				correctamente, con manejo de concurrencia y comunicación gRPC.	con gRPC	
			Rango del punteo	10-6	6-0	
1.4	**Arquitectura General Despliegue en Zot**	10	Descripción	Despliegue completo y funcional de todos los componentes en GCP. Despliegue correcto de Zot en vm para imágenes.	Zot no desplegado en Vm. Zot no almacena correctamente las imágenes	
			Rango del punteo	10-6	6-0	
1.5	**Message Broker y Consumidor RabbitMQ**	15	Descripción	RabbitMQ desplegado y configurado. El Writer publica mensajes correctamente. Los consumidores leen las colas/tópicos y procesan los mensajes.	Incorrecta implementación de la cola. Los writes publican incorrectamente los mensajes. Los consumidores no leen/procesan correctamente las colas	
			Rango del punteo	15-9	9-0	
1.6	**Almacenamiento (Valkey)**	20	Descripción	Valkey desplegado con KubeVirt. Los consumidores almacenan datos en Valkey.	Valkey no desplegado	

			Rango del punteo	20-6	5-0	
1.7	**Grafana**	25	Descripción	Grafana desplegado. Visualización de los datos requeridos en Grafana.	. Dashboard sin toda la información requerida	
			Rango del punteo	25-10	10-0	
	Puntos	**90**				
TOTAL		**90**				
2	**Análisis, Documentación y Cumplimiento**					
2.1	**Pruebas de Carga, HPA y Análisis de Réplicas**	5	Descripción	Locust genera la carga especificada. la API Rust funciona según lo definido. Se realizan y documentan pruebas comparativas con 1 y 2 réplicas para Go Writers y Valkey, analizando rendimiento.	Falta de implementación del Locust. Falta de documentación de las pruebas comparativas	
			Rango del punteo	5-3	2-0	

			Descripción	El manual es claro, completo, incluye todas las secciones requeridas (despliegue, arquitectura, pruebas de funcionalidad).	Manual sin todas las secciones requeridas	
2.2	**Manual tecnico**	2	Rango del punteo	2-1	0	
			Descripción	Se responden de forma correcta y justificada todas las preguntas planteadas durante la calificación.	No se responden las preguntas de forma clara	
2.2	**Respuestas a Preguntas**	3	Rango del punteo	3-2	1-0	
	Puntos	**10**				
TOTAL		**10**				
TOTAL FINAL		**100**				